INFORME DE LABORATORIO 3

Diseño y validación de gramáticas libres de contexto

Antecedente	Información
Asignatura	Teoría de la Computación - INFO1148
Carrera	Ingeniería Civil en Informática
Docente	Marcos Lévano
Integrante	Álvaro Valenzuela Paillacar
Fecha	21 de octubre de 2026

1. Introducción

En este laboratorio diseño una gramática libre de contexto para un lenguaje de programación simplificado. Identifico sus símbolos, defino las producciones y compruebo asignaciones, operaciones aritméticas y salidas mediante derivaciones y árboles generados con JFLAP 7.1. La validación distingue instrucciones de programas completos y detecta errores sintácticos. Como extensión incorporo declaraciones de variables enteras y reales.

2. Desarrollo

2.1 Pregunta 1 - Componentes de la gramática

a) Símbolos terminales

Categoría	Elementos identificados
Palabras reservadas	Begin, End, Programa, salida. Extensión: entero, real.
Operadores	Aritméticos: +, -, *, /, %. Asignación: =.
Símbolos especiales	# como terminador de instrucción.
Identificadores	Token id. Patrón: [a-zA-Z][a-zA-Z0-9_]*, excluyendo palabras reservadas. Ejemplos: x, z2, temporal, operativo_01_02.
Números	Token num. Patrón: [0-9]+(\.[0-9]+)?. Enteros o decimales no negativos: 20, 0, 1.345.

b) No terminales, c) símbolo inicial y d) definición formal

G = (V, Σ, P, S), con S = <programa>.

V = {<programa>, <lista_instrucciones>, <instruccion>, <asignacion>, <asig_simple>, <asig_expr>, <expresion>, <valor>, <operador>, <salida>, <declaracion>, <tipo>}.
Σ = {Begin, End, Programa, salida, entero, real, +, -, *, /, %, =, #, id, num}.
P es el conjunto de producciones de la sección 2.2; V ∩ Σ = ∅.

e) Análisis

id y num son categorías léxicas, no listas de valores. Una gramática finita puede generar un lenguaje infinito, pero enumerar individualmente todos los identificadores y números requeriría infinitas alternativas. Los patrones léxicos permiten reconocer sus lexemas y conservar una gramática sintáctica finita.

2.2 Pregunta 2 - Construcción de la GLC

a) Producciones propuestas

No terminal	Lado derecho de la producción
<programa>	Begin Programa id <lista_instrucciones> End
<lista_instrucciones>	<instruccion> <lista_instrucciones> | <instruccion>
<instruccion>	<asignacion> | <salida> | <declaracion>
<asignacion>	<asig_simple> | <asig_expr>
<asig_simple>	id = <valor> #
<asig_expr>	id = <expresion> #
<expresion>	<valor> <operador> <valor>
<valor>	id | num
<operador>	+ | - | * | / | %
<salida>	salida id #
<declaracion>	<tipo> id #
<tipo>	entero | real

Cada fila representa A → α. Al separar las alternativas se obtienen 22 producciones. Cada lado izquierdo contiene un único no terminal, por lo que G es libre de contexto. Las dos últimas filas y la alternativa <declaracion> constituyen la extensión.

b) Función de los no terminales

No terminal	Función
<programa>	Delimita el programa e incluye su nombre y cuerpo.
<lista_instrucciones>	Recursión por la derecha para una o más sentencias; no permite cuerpo vacío.
<instruccion>	Selecciona asignación, salida o declaración.
<asignacion>	Agrupa las dos clases de asignación.
<asig_simple> / <asig_expr>	Distinguen un valor directo de una operación binaria.
<expresion> / <valor>	Describen la operación infija y sus operandos id o num.
<operador> / <salida>	Seleccionan el operador y definen salida id #.
<declaracion> / <tipo>	Introducen la variable y el tipo entero o real.

c) Verificación del programa de referencia completo

El programa de referencia es representable después del análisis léxico. Su encabezado Begin Programa operativo_01_02 corresponde a Begin Programa id y su cierre es End. El cuerpo contiene 20 sentencias: 6 asignaciones simples, 7 operaciones y 7 salidas.

Sentencias del programa	Cantidad	Producción o elección
x = 1.345 #; y = 20 #; temporal = 0 #; temporal = x #; x = y #; y = temporal #	6	<asig_simple> → id = <valor> #
z = x + y #; z2 = x + y #	2	<asig_expr>, <operador> → +
h = z - x #; h2 = z - x #	2	<asig_expr>, <operador> → -
k = h * y #	1	<asig_expr>, <operador> → *
t = k / z #	1	<asig_expr>, <operador> → /
m = k % 10 #	1	<asig_expr>, <operador> → %
salida z #; salida h #; salida k #; salida t #; salida m #; salida x #; salida y #	7	<salida> → salida id #
Total	20	19 expansiones recursivas de la lista y un caso base.

Esquema de la derivación del programa

```
<programa>
⇒ Begin Programa id <lista_instrucciones> End
⇒* Begin Programa id <instruccion>^19 <lista_instrucciones> End
⇒ Begin Programa id <instruccion>^20 End
⇒* Begin Programa id id = num # id = num # … salida id # End
```

La potencia expresa repetición en este esquema explicativo, no un nuevo símbolo de la gramática. Al expandir cada instrucción en el orden del archivo programa_referencia.txt, se obtiene su secuencia completa de 91 tokens, aceptada por el verificador incluido.

Comentarios y separación entre léxico y sintaxis

El comentario /* operaciones y salidas */ es reconocido y descartado por el analizador léxico, junto con los espacios y saltos de línea. Su patrón es /*[\s\S]*?*/: admite contenido multilínea, sin anidamiento. El comentario no se cuenta como sentencia ni se añade a Σ. Así, un cuerpo formado únicamente por comentarios no satisface la exigencia de una o más instrucciones.

Los lexemas operativo_01_02, z2 y temporal cumplen el patrón de id; 1.345, 20, 10 y 0 cumplen el de num. Las palabras reservadas se reconocen antes que id. La gramática describe sintaxis: no comprueba declaración previa, compatibilidad de tipos, valores de variables ni división por cero, que corresponden al análisis semántico.

2.3 Pregunta 3 - Derivaciones y árbol sintáctico

a) Derivación por la izquierda de z = x + y

Se parte de <instruccion> para analizar una sentencia aislada. En cada paso se sustituye únicamente el no terminal más a la izquierda. La derivación formal termina en tokens.

Paso	Forma sentencial	Producción aplicada
0	<instruccion>	Inicio
1	<asignacion>	<instruccion> → <asignacion>
2	<asig_expr>	<asignacion> → <asig_expr>
3	id = <expresion> #	<asig_expr> → id = <expresion> #
4	id = <valor> <operador> <valor> #	<expresion> → <valor> <operador> <valor>
5	id = id <operador> <valor> #	<valor> → id
6	id = id + <valor> #	<operador> → +
7	id = id + id #	<valor> → id

Asociación léxica externa: los tres tokens id tienen lexemas z, x e y, respectivamente. Por tanto, la secuencia de tokens corresponde a z = x + y #. Esta asociación no es una producción de G.

b) Derivación por la izquierda de salida z

Paso	Forma sentencial	Producción aplicada
0	<instruccion>	Inicio
1	<salida>	<instruccion> → <salida>
2	salida id #	<salida> → salida id #

Asociación léxica externa: el token id tiene lexema z. La instrucción fuente es salida z #.

Como S = <programa>, ninguna instrucción aislada pertenece por sí sola a L(G). Se incorpora a un programa mediante <programa> ⇒ Begin Programa id <lista_instrucciones> End ⇒ Begin Programa id <instruccion> End y la derivación de la instrucción.

Derivación por la izquierda del programa envolvente en JFLAP

```
S
⇒ bpiLf
⇒ bpiIf
⇒ bpiAf
⇒ bpiCf
⇒ bpii=E#f
⇒ bpii=VOV#f
⇒ bpii=iOV#f
⇒ bpii=i+V#f
⇒ bpii=i+i#f
```

Esta es la codificación del programa Begin Programa demo z = x + y # End. Las cuatro ocurrencias de i conservan, en orden, los lexemas demo, z, x e y. La tabla de equivalencias del anexo A permite leer los símbolos de un carácter utilizados por JFLAP.

c) Árbol de derivación en JFLAP

El árbol siguiente corresponde al programa Begin Programa demo z = x + y # End. El subárbol con raíz I es exactamente el de la instrucción pedida. Se obtuvo con Gramatica_Lab3.jff, cuya producción inicial es S → bpiLf, y la entrada bpii=i+i#f.

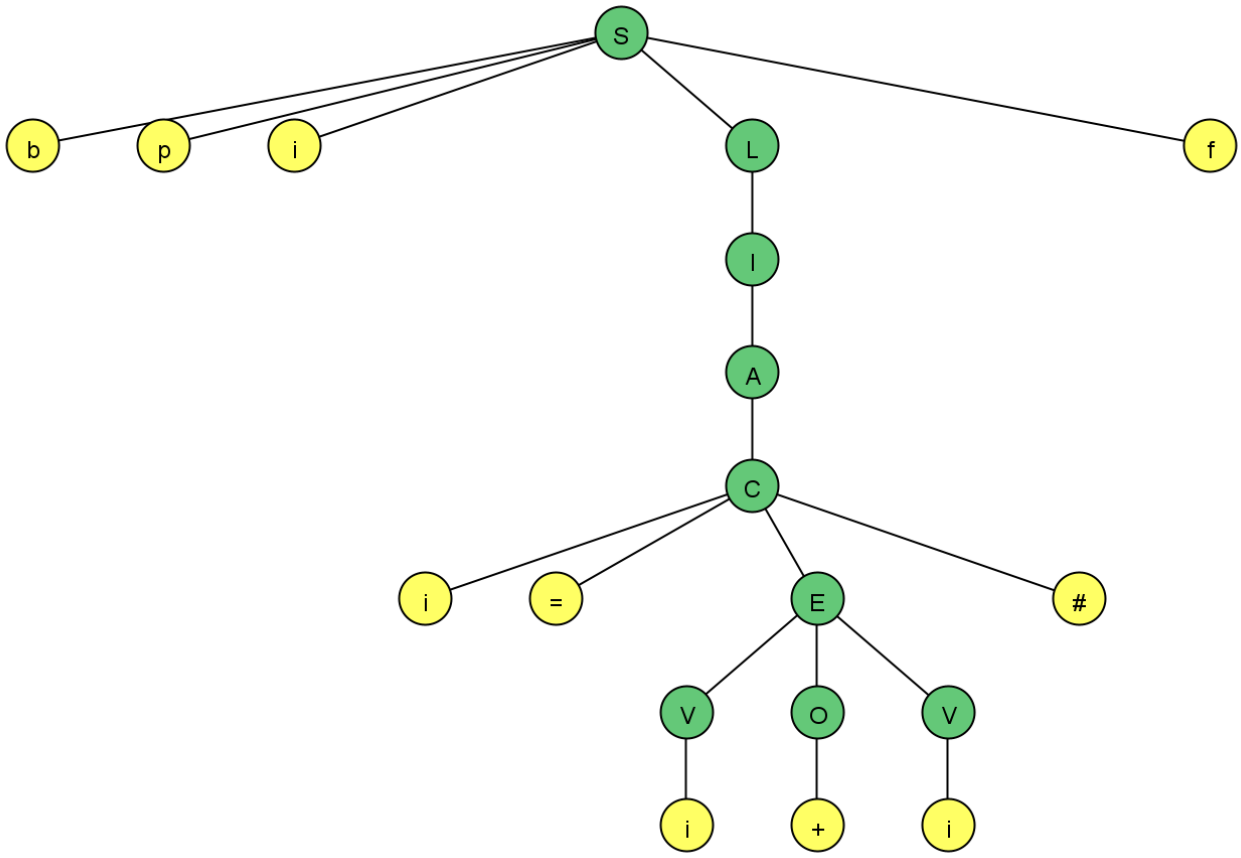


Figura 1. Árbol generado por el analizador y el panel nativo de JFLAP 7.1. Verde: no terminales; amarillo: terminales.

Símbolo JFLAP	Equivalencia
S, L, I	<programa>, <lista_instrucciones>, <instruccion>
A, C, E, V, O	<asignacion>, <asig_expr>, <expresion>, <valor>, <operador>
b, p, f, i	Begin, Programa, End, id; cada i conserva su lexema asociado.

d) Explicación del árbol

La raíz S representa el programa completo. Sus hijos b, p, i, L y f aplican S → bpiLf. El nodo L produce I; el subárbol I → A → C representa la asignación con expresión. C tiene hijos i, =, E, #; E tiene hijos V, O, V; y estos producen i, +, i. Cada nodo interno representa la aplicación de una producción.

Los terminales son hojas. En el subárbol I, las hojas de izquierda a derecha son i = i + i #, equivalentes a id = id + id # y asociadas a z = x + y #. En el árbol completo se añaden las hojas del encabezado y del cierre. La secuencia de expansiones por la izquierda del apartado a) produce este mismo subárbol.

e) Validación de las tres cadenas solicitadas

Cadena	¿Puede generarse?	Justificación
x = 20 #	Sí, desde <instruccion>.	<instruccion> ⇒ <asignacion> ⇒ <asig_simple> ⇒ id = <valor> # ⇒ id = num #. Lexemas: x y 20.
z = x * y #	Sí, desde <instruccion>.	Derivación del apartado a), usando <operador> → *. Tokens: id = id * id #.
salida temporal #	Sí, desde <instruccion>.	<instruccion> ⇒ <salida> ⇒ salida id #. El lexema temporal es un identificador válido.

Vista de la instrucción aislada en JFLAP

La variante auxiliar permite analizar una sentencia sin añadir encabezado y cierre. Solo sustituye la producción inicial por S → I; las otras 21 producciones coinciden con el archivo principal. En este archivo, S es una raíz auxiliar y no representa <programa>.

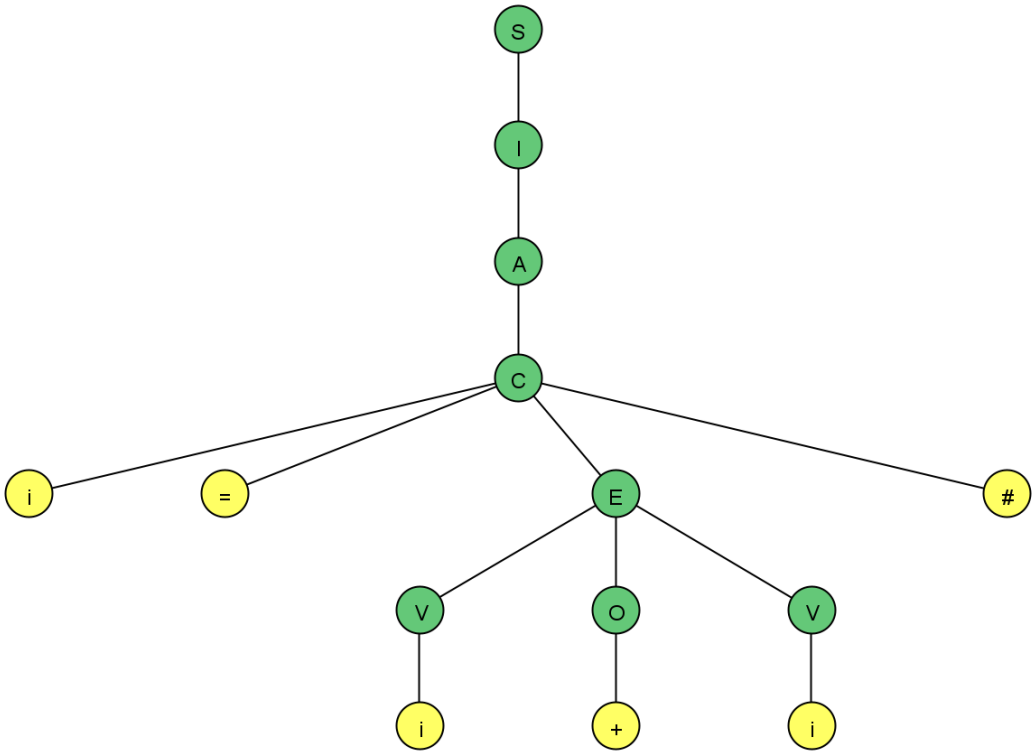


Figura 2. Árbol nativo de la instrucción i=i+i#, con el envoltorio auxiliar S → I. El subárbol I reproduce la derivación de z = x + y #.

2.4 Pregunta 4 - Validación y detección de errores

Los casos 1 a 3 se analizan como instrucciones. El caso 8 se analiza desde <programa>; las barras del enunciado indican cambios de línea y no son tokens /. Se distingue así la validez de una sentencia de la pertenencia a L(G).

Caso y cadena	Resultado	Justificación o error detectado
1. x = 10 #	Válida como instrucción.	<asig_simple> → id = <valor> #, con <valor> → num. En JFLAP: i=d#.
2. z = x + y #	Válida como instrucción.	<asig_expr> → id = <expresion> #; expresión binaria infija. En JFLAP: i=i+i#.
3. salida z #	Válida como instrucción.	<salida> → salida id #. En JFLAP: si#.
4. x 10 #	Inválida.	Tras el identificador debe aparecer =. Falta el operador de asignación. En JFLAP: id# se rechaza.
5. z = + x y #	Inválida.	El primer operando debe ir después de =; el operador + está antes de ambos operandos. Corregir a z = x + y # moviendo el operador. En JFLAP: i=+ii# se rechaza.
6. salida #	Inválida.	Falta el identificador entre salida y #. En JFLAP: s# se rechaza.
7. x = 20	Inválida.	Falta el terminador #. En JFLAP: i=d se rechaza.
8. Begin Programa prueba x = 5 # salida x # End	Válida como programa completo.	Encabezado, asignación, salida y cierre correctos. En JFLAP: bpii=d#si#f, aceptada con la gramática principal.

Derivación del caso 8

```
<programa>
⇒ Begin Programa id <lista_instrucciones> End
⇒ Begin Programa id <instruccion> <lista_instrucciones> End
⇒* Begin Programa id id = num # <lista_instrucciones> End
⇒ Begin Programa id id = num # <instruccion> End
⇒* Begin Programa id id = num # salida id # End
```

Las flechas ⇒* resumen expansiones por la izquierda de asignación y salida ya desarrolladas. Asociación léxica, en orden: el nombre es prueba; los identificadores de las sentencias son x y x; num tiene lexema 5.

Análisis general

Los errores corresponden a omisión de terminales obligatorios (= o #), omisión del identificador requerido por salida y posición incorrecta del operador aritmético. La gramática detecta estas fallas por la estructura de sus producciones; no necesita evaluar los valores de las variables. Los doce ensayos nativos de JFLAP, incluidos los ocho casos y cuatro de la extensión, coinciden con estas clasificaciones.

3. Extensión de la gramática

Construcción seleccionada y nueva sintaxis

Selecciono la opción C: declaración de variables. Una declaración tiene la forma <tipo> id #, con tipo entero o real. Por ejemplo, entero x # y real temperatura #. Se admite como una instrucción en cualquier posición del cuerpo; exigir declaración previa o impedir duplicados requeriría análisis semántico.

Elemento	Modificación
Nuevos terminales	entero y real, reconocidos como palabras reservadas y excluidos de id.
Nuevos no terminales	<declaracion> y <tipo>.
Producciones añadidas	<instruccion> → <declaracion> <declaracion> → <tipo> id # <tipo> → entero | real

Cadena válida: real temperatura #. Cadena inválida: entero #, porque falta el identificador. También se rechazan real temperatura por ausencia de # y x entero # por orden incorrecto.

Derivación por la izquierda de la cadena válida

```
<instruccion> ⇒ <declaracion> ⇒ <tipo> id # ⇒ real id #
```

El lexema del token id es temperatura. La asociación se realiza fuera de la derivación formal. Para un programa completo: <programa> ⇒ Begin Programa id <lista_instrucciones> End ⇒ Begin Programa id <instruccion> End ⇒ Begin Programa id <declaracion> End ⇒ Begin Programa id <tipo> id # End ⇒ Begin Programa id real id # End. Los lexemas de los dos id pueden ser demo y temperatura.

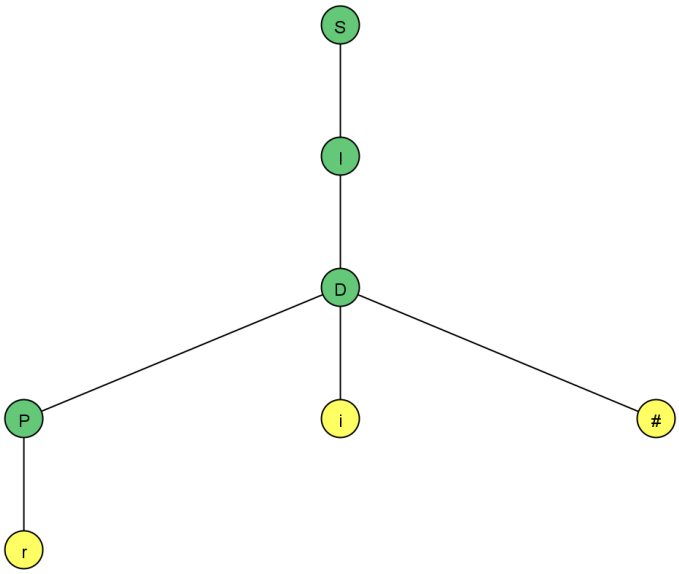


Figura 3. JFLAP, entrada ri#. En el archivo auxiliar S → I reemplaza solo la producción inicial del archivo principal; S es aquí la raíz auxiliar, no un programa. D equivale a <declaracion>, P a <tipo>, r a real e i a id.

La extensión mantiene válidos los programas originales sin declaraciones. No se agregan reglas semánticas de tipos a la GLC.

4. Conclusiones

El diseño permite representar las estructuras requeridas con una gramática finita y coherente. La separación entre tokens y lexemas evita confundir el análisis léxico con las producciones sintácticas. La recursión de la lista asegura una o más sentencias, mientras que la separación entre asignación simple, operación y salida facilita derivar y validar cada construcción.

Las derivaciones por la izquierda y los árboles de JFLAP muestran la misma estructura. La comprobación de casos válidos e inválidos permite localizar símbolos faltantes y operadores mal ubicados. La extensión de declaraciones se integra mediante dos terminales y dos no terminales nuevos, conservando el lenguaje base. El alcance del modelo es sintáctico: las condiciones de uso de variables y tipos corresponden a una etapa semántica posterior.

5. Consideraciones de entrega

El informe se entrega en PDF, acompañado de la gramática principal, una variante auxiliar para instrucciones, las capturas de los paneles y árboles nativos de JFLAP y los resultados de validación. El repositorio contiene los archivos correspondientes:

https://github.com/MelonCaja/INFO1148-Laboratorio-3

Archivo	Uso
Gramatica_Lab3.jff	Archivo principal: S → bpiLf. Analiza programas completos.
Gramatica_Lab3_Instrucciones.jff	Variante auxiliar: solo cambia S → I para analizar una instrucción aislada. Las otras 21 producciones son idénticas.
evidencias/	Cuatro árboles, cuatro capturas de paneles, trazas de derivación y resultados.
programa_referencia.txt	Programa completo de referencia, con el comentario original.
verificar_entrega.py	Verificador independiente sobre los JFF reales, ejecutable con Python 3 sin dependencias externas.

Resumen de validación reproducible

Comprobación	Resultado
Lectura y escritura con XMLCodec de JFLAP	Dos archivos; 22 producciones en cada uno, sin pérdida.
Analizador nativo BruteParser de JFLAP	12 casos; resultados esperados en todos.
Reconocedor independiente Earley sobre los JFF	4362 cadenas: 25 aceptadas y 4337 rechazadas; cero discrepancias con un clasificador independiente.
Programas de dos instrucciones	625 combinaciones aceptadas.
Programa de referencia y casos adicionales	Programa completo aceptado; 10 casos adicionales correctos, incluido rechazo de cuerpo vacío o con solo comentarios.

Las pruebas finitas respaldan el diseño y complementan el análisis de las producciones; no sustituyen una demostración sobre todas las cadenas posibles.

Anexo A - Equivalencias y reproducción en JFLAP

A.1 Correspondencia de símbolos

No terminal formal	JFLAP	No terminal formal	JFLAP
<programa>	S	<lista_instrucciones>	L
<instruccion>	I	<asignacion>	A
<asig_simple>	B	<asig_expr>	C
<expresion>	E	<valor>	V
<operador>	O	<salida>	Y
<declaracion>	D	<tipo>	P

Token formal	JFLAP	Token formal	JFLAP
Begin	b	End	f
Programa	p	salida	s
entero	n	real	r
id	i	num	d
+, -, *, /, %, =, #	Sin cambio	Espacios y comentarios	Se omiten en el léxico.

A.2 Procedimiento para reproducir las evidencias

1. Abrir Gramatica_Lab3.jff en JFLAP 7.1 mediante File → Open.
2. Elegir Input → Brute Force Parse e introducir bpii=i+i#f, sin espacios.
3. Iniciar el análisis y avanzar con Step hasta completar la derivación; la vista Noninverted Tree muestra el árbol de la figura 1.
4. Para instrucciones aisladas, abrir Gramatica_Lab3_Instrucciones.jff y usar i=i+i#, si# o ri#. El nodo superior S aplica S → I y actúa solo como envoltorio auxiliar.
5. Usar bpii=d#si#f con el archivo principal para reproducir el caso 8.

A.3 Producciones codificadas del archivo principal

Variable	Producciones
S	S → bpiLf
L	L → IL | I
I	I → A | Y | D
A	A → B | C
B / C	B → i=V#; C → i=E#
E / V	E → VOV; V → i | d
O	O → + | - | * | / | %
Y / D / P	Y → si#; D → Pi#; P → n | r

Para ejecutar la comprobación independiente desde la carpeta de entrega: python verificar_entrega.py. Sus resultados se escriben en evidencias/verificacion.json.

Anexo B - Capturas de los paneles de JFLAP

Las capturas muestran componentes nativos de JFLAP 7.1 renderizados programáticamente, con el árbol calculado por su propio analizador. No se utilizó Graphviz para las figuras de esta entrega. Las trazas automáticas de JFLAP pueden expandir varios símbolos en un paso; las derivaciones por la izquierda de la sección 2.3 se presentan por separado.

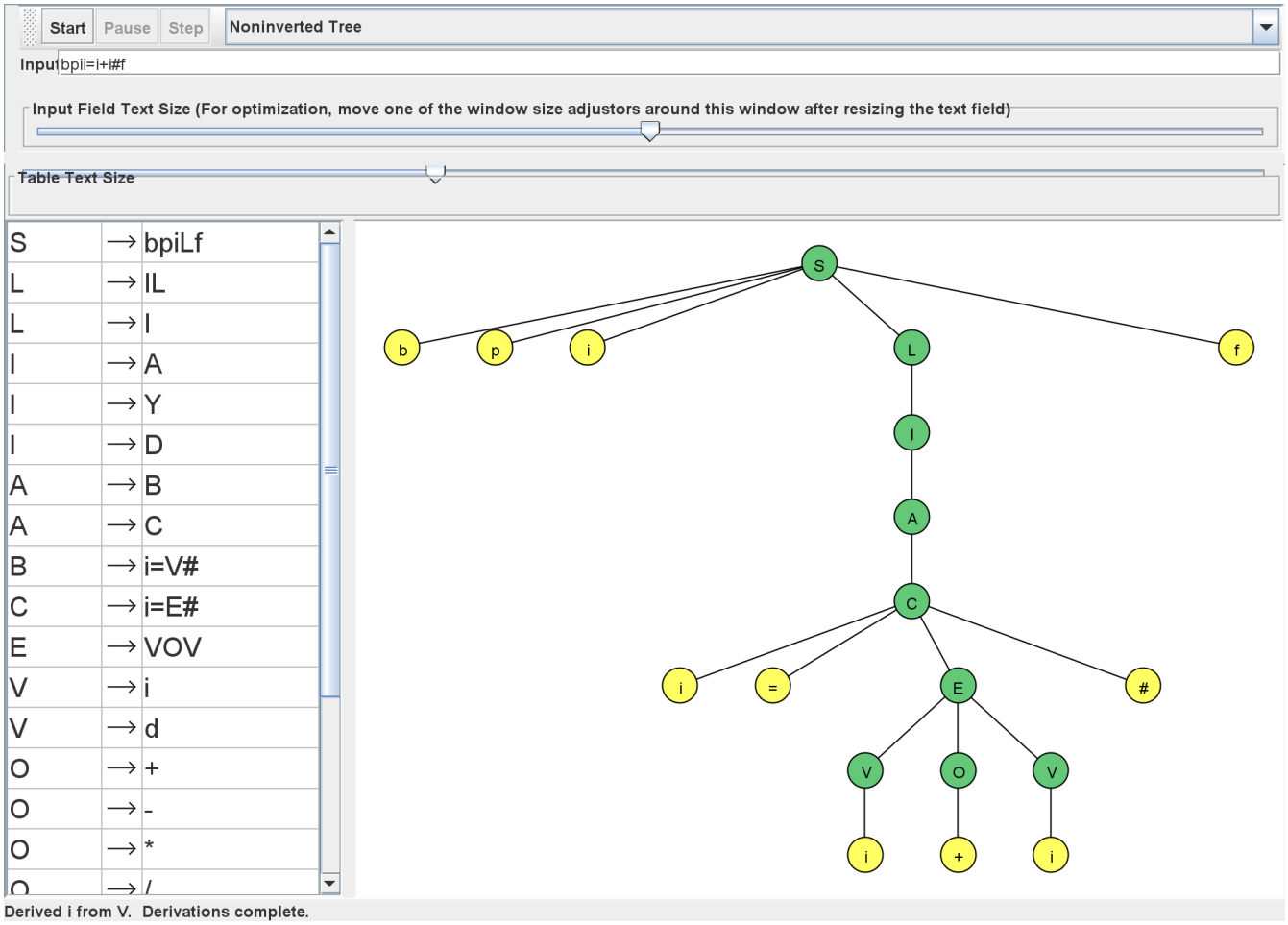


Figura 4. Panel nativo de análisis del archivo principal. Entrada bpii=i+i#f y árbol completo con raíz S.

Anexo B - Panel de la instrucción aislada

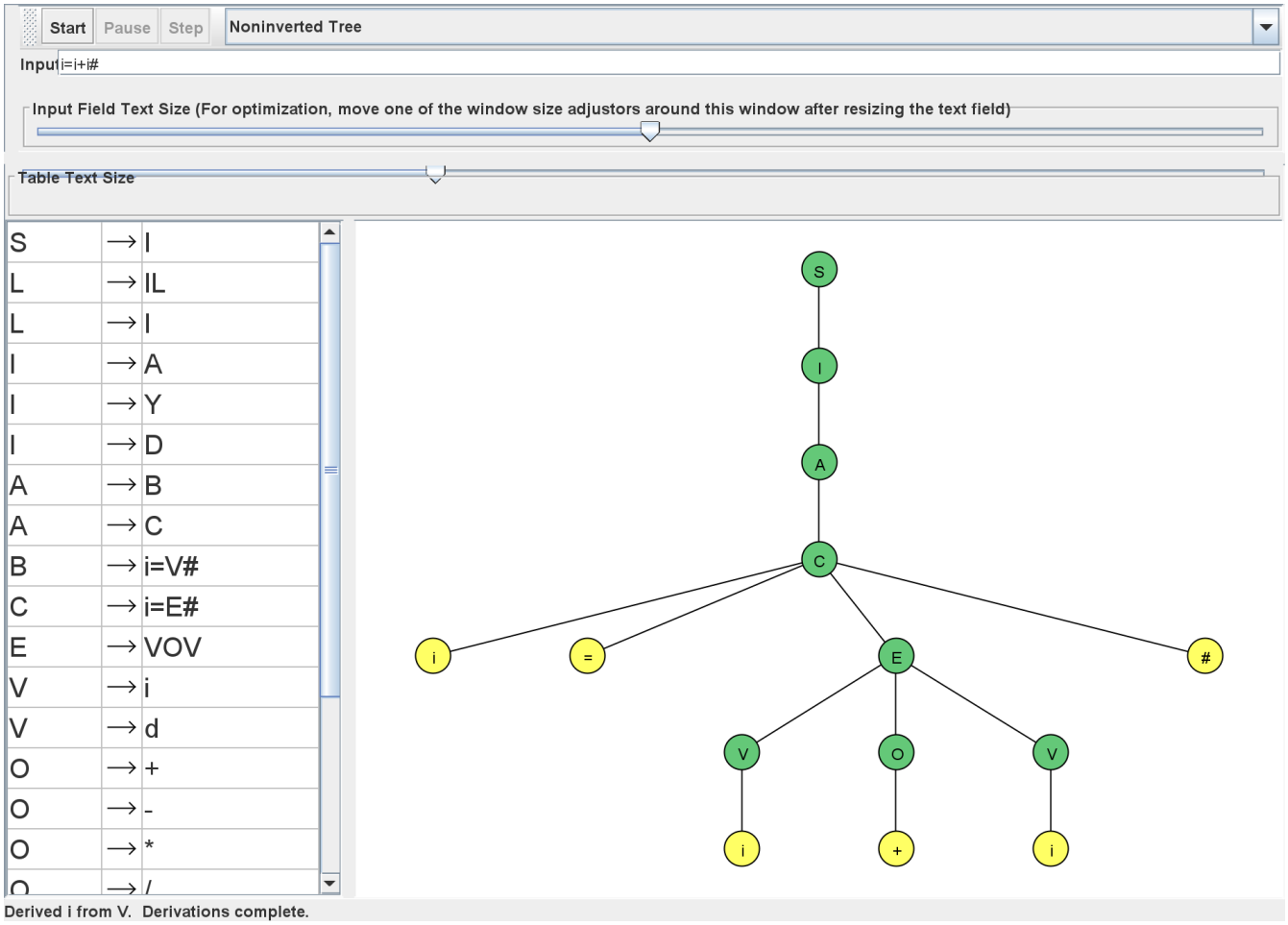


Figura 5. Panel nativo del archivo auxiliar. Entrada i=i+i# y árbol de la asignación, precedido del envoltorio S → I.

Las hojas amarillas del subárbol I se leen como i=i+i#. Las apariciones de i representan los lexemas z, x e y en el ejemplo solicitado. La captura incluye las producciones utilizadas y el estado de derivación completa.

Anexo C - Evidencia de salida y pruebas nativas

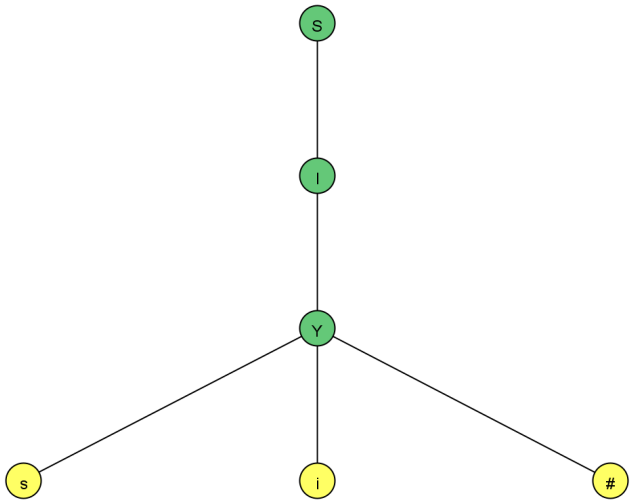


Figura 6. Árbol nativo de salida, entrada si#. En la variante auxiliar S → I; luego I → Y y Y → si#.

C.1 Resultados del analizador nativo de JFLAP

Caso	Archivo	Entrada	Resultado
1	Auxiliar	i=d#	Aceptada
2	Auxiliar	i=i+i#	Aceptada
3	Auxiliar	si#	Aceptada
4	Auxiliar	id#	Rechazada
5	Auxiliar	i=+ii#	Rechazada
6	Auxiliar	s#	Rechazada
7	Auxiliar	i=d	Rechazada
8	Principal	bpii=d#si#f	Aceptada
Extensión 1	Auxiliar	ni#	Aceptada
Extensión 2	Auxiliar	ri#	Aceptada
Extensión 3	Auxiliar	n#	Rechazada
Extensión 4	Auxiliar	rii#	Rechazada

El archivo pruebas_jflap.csv registra el resultado esperado y el obtenido. El generador Java conserva además las derivaciones automáticas y permite regenerar las imágenes a partir de JFLAP7.1.jar.